

Hybrid Parallel Distributed Password Cracking System using MPI, OpenMP and CUDA (C++)

A Project Component for
Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Divyam Puri	102483023
2	Ananya Kumar	102303598
3	Lavish Arora	102483041
4	Bharti Verma	102303575

Under the guidance of
Dr. Saif Nalband
(Assistant Professor, DCSE)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2025

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Introduction to Problem Statement	1
2	Literature Review	3
3	Research Gaps	6
4	Problem Formulation	8
4.1	Mathematical Representation of the Search Space	8
4.2	Formal Problem Statement	8
4.3	Computational Bottlenecks in Hybrid Systems	9
4.3.1	The Data Starvation Problem	9
4.3.2	Asynchronous Success Propagation	9
4.4	Distributed Workload Coordination	9
4.5	Performance Scalability Objective	10
5	Objectives	10
6	Methodology	12
6.1	System Overview	12
6.2	Worker Execution	13
6.3	Index-Based Password Candidate Generation	13
6.4	Dictionary Attack Methodology	13
6.5	Hash Computation and Result Collection	14
6.6	Performance Evaluation	14
6.7	Block Diagram	15
6.8	Pseudocode	16
7	Datasets	17

8	Results and Discussion	18
8.1	Experimental Results	18
8.1.1	Performance Summary	18
8.1.2	Password Cracking Time Comparison	18
8.2	Analysis of Results	19
8.2.1	Impact of Parallelism	19
8.2.2	Scalability	19
8.2.3	Efficiency of Attack Modes	19
8.2.4	Resource Utilization	20
8.3	Discussion	20
8.4	Conclusion from Results	21

List of Figures

1	Password Cracking Methodology	2
2	Hybrid MPI–OpenMP–CUDA Password Cracking Methodology	15

List of Tables

1	Summary of Literature Review	3
2	Datasets	17
3	Performance Summary	18
4	Password Cracking Time Comparison	18

1 Introduction

1.1 Background

Passwords are the primary security mechanism for protecting digital systems, but weak passwords remain vulnerable to brute force and dictionary attacks. Password cracking requires testing millions of combinations through repeated cryptographic hash computations, which is extremely time-consuming on traditional CPU-based systems. This project proposes a high-performance hybrid password cracking framework implemented entirely in C++ that utilizes modern parallel computing technologies to accelerate the process. The system integrates MPI for distributed execution across multiple machines, OpenMP for multi-core CPU parallelism, and CUDA for GPU-accelerated hashing to achieve maximum computational efficiency.

1.2 Introduction to Problem Statement

The rapid proliferation of digital systems and online services has significantly increased reliance on password-based authentication mechanisms for securing sensitive information. However, the growing computational capabilities available to adversaries have exposed inherent vulnerabilities in traditional password security models, particularly against brute-force and dictionary-based attacks. Password cracking, while often associated with malicious intent, also serves as a critical tool in cybersecurity for auditing password strength, recovering lost credentials, and evaluating system resilience. The fundamental challenge in this domain lies in the immense size of the password search space, which leads to prohibitively high computational time when using conventional sequential or even moderately parallel approaches.

To address this limitation, there is a need for scalable and high-performance computational frameworks capable of efficiently exploring large search spaces. Parallel and distributed computing paradigms offer a viable solution by enabling simultaneous processing across multiple computational units. However, existing implementations often rely on a single level of parallelism—either multi-core CPUs, distributed clusters, or GPUs—thereby underutilizing the full spectrum of available hardware resources. This project seeks to bridge this gap by designing and implementing a hybrid parallel dis-

tributed password cracking system that integrates message passing, shared-memory parallelism, and GPU acceleration. The objective is to demonstrate how a multi-layered parallel architecture can significantly enhance computational throughput and achieve substantial speedup, thereby making large-scale password analysis more efficient and scalable.

Figure 1 shows the methodology we have followed.

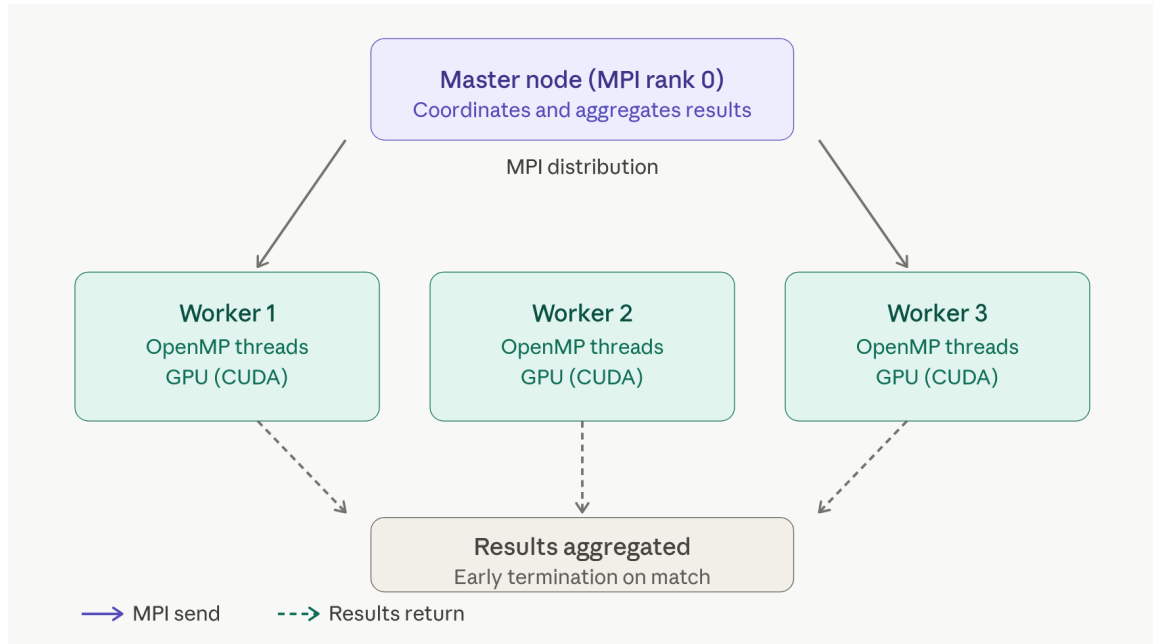


Figure 1: Password Cracking Methodology

2 Literature Review

Table 1: Summary of Literature Review

References	Concepts Highlighted	Technique(s) Used	Significance/ Proposal	Limitations
[?]	GPU-based parallel password generation; Probabilistic Context-Free Grammars (PCFG); Time-based cracking evaluation; Integration of data-driven models with Hashcat; Memory load balancing in GPU tasks	Memory load balancing storage structure for PCFG; Parallel candidate generation algorithm; Two-dimensional lookup table for thread assignment; Asynchronous CPU-GPU pipeline collaboration; Container smoothing and probability re-assignment	Parallel_PCFCG achieves 30–60% cracking rate within 30 minutes; Requires only 14.33% of the time of best prior models to match their cracking rates; Bridges academic PCFG models with industrial tool Hashcat; Introduces time-based metrics as a more realistic evaluation framework	Evaluated only on generic offline cracking; targeted and masked scenarios excluded; Full potential reached with a single GPU — multi-GPU scaling not addressed; Smoothing technique introduces slight probability ordering distortion; Neural network models not compared due to time constraints

[?]	Brute force vs. dictionary attack comparison; Parallel computing for password cracking; GPU hardware impact on cracking speed; Effect of special characters on cracking difficulty; Multi-core CPU and GPU parallelism	Python <code>ProcessPoolExecutor</code> for brute force; Python <code>multiprocessing</code> module for dictionary attack; OpenMP in C++ with static and dynamic scheduling; Hashcat v6.2.6 with CUDA; MD5 hash encryption; RockYou dataset (14.4M passwords)	CUDA GPU (GTX 1050 Ti) is 11.5× faster than integrated GPU for brute force; Dictionary attack is approximately 930× faster than brute force for complex passwords; 4.4× speedup achieved with 8-core static OpenMP scheduling; Demonstrates that special characters substantially increase cracking time	Small test set of only 10 passwords limits generalisability; Python dictionary attack showed no parallelism improvement due to process management overhead; Only MD5 encryption tested — no SHA-256, bcrypt, or similar; Hardware limited to consumer-grade GPUs; No probabilistic or data-driven models considered
-----	--	---	--	---

[?]	HPC cluster construction for forensic use; MPI-based parallel password cracking; Brute force and dictionary attacks on Linux shadow files; Password complexity and combinatorial analysis; Computer forensics applications of parallel computing	Beowulf HPC cluster with 12 nodes and dual AMD Opteron CPUs; Message Passing Interface (MPI) via MPICH 1.2.6; Custom brutest application in ANSI-C; Root node assigns work vectors to slave nodes; MD5-based crypt() hashing for Linux shadow password files; Trigram-based brute force to reduce inter-node traffic	Demonstrated approximately 12,500 hashes per second on a 10-node cluster; Showed that an affordable COTS cluster (~€18,000) can support forensic parallel computing; Established foundational work for parallel forensic analysis tools; Highlighted fundamental limits of brute force even with parallelism	Very low throughput (12,500 hashes/sec) compared to modern GPU solutions; No GPU acceleration — purely CPU and cluster based; No scalability measurements across different node counts conducted; Limited forensic relevance when physical disk access is available; Feasibility study only — no production-level optimisation performed
-----	---	---	---	---

3 Research Gaps

Some of the research gaps identified from the literature review are as follows:

- ***Absence of a Unified Tri-Layer Parallel Architecture:*** Prior works address parallelism in isolation — Bengtsson (2007) uses only MPI-based CPU clustering, Alkhwaja et al. (2023) treat CPU multiprocessing and GPU acceleration as separate experiments, and Xu et al. (2025) confine their contribution to single-GPU parallelism. No existing work integrates MPI, OpenMP, and CUDA concurrently within a single system, which the proposed architecture directly addresses.
- ***Lack of GPU Acceleration Within Distributed Node Architectures:*** Bengtsson (2007) constructs a multi-node cluster yet relies entirely on CPU processing, achieving only $\sim 12,500$ hashes per second. Xu et al. (2025) demonstrate strong GPU-level acceleration but do not scale across distributed nodes. The proposed system deploys CUDA kernels on every MPI worker node simultaneously, achieving throughput that scales with both node count and per-node GPU capacity.
- ***Absence of Coordinated Early Termination in Distributed Settings:*** None of the reviewed works systematically address the propagation of a success signal across all distributed nodes upon discovery of the target password, resulting in redundant computation. The proposed system introduces an explicit MPI broadcast termination protocol that halts all worker processes promptly upon success.
- ***Static and Non-Adaptive Work Distribution:*** All three reviewed works employ static, equal-chunk work distribution without accounting for differences in node throughput. Bengtsson (2007) explicitly acknowledges the absence of dynamic load balancing. The proposed system implements a structured MPI communication protocol designed to accommodate future integration of dynamic scheduling and work-stealing strategies.
- ***Fragmented Treatment of Attack Modalities:*** Alkhwaja et al. (2023) compare brute force and dictionary attacks across different languages and tools, introducing confounding variables. Xu et al. (2025) consider only probabilistic candidate

generation. The proposed system unifies both attack modes within the same MPI–OpenMP–CUDA pipeline, enabling consistent evaluation under identical hardware conditions.

- ***Narrow and Inconsistent Performance Evaluation:*** Xu et al. (2025) restrict benchmarking to the PCFG domain, Alkhwaja et al. (2023) test only 10 passwords, and Bengtsson (2007) conducts no scalability measurements across node counts. The proposed system addresses this through a structured benchmark suite evaluating single-threaded, OpenMP, MPI, and fully hybrid configurations systematically.

4 Problem Formulation

The fundamental challenge addressed in this project is the optimization of a multi-dimensional search space for cryptographic hash reversal under strict resource and time constraints. In a typical brute-force scenario, the workload is computationally bound, requiring a design that eliminates idle cycles across distributed nodes. This section formalizes the computational complexity and the technical bottlenecks inherent in distributed password recovery systems.

4.1 Mathematical Representation of the Search Space

The search space is defined by the set of all possible permutations of a given character set C within a length range $[min, max]$. Let L be the password length. The total search space S is defined by the cardinality:

$$|S| = \sum_{i=min}^{max} |C|^i \quad (1)$$

For an 8-character alphanumeric password ($|C| = 62$), $|S| \approx 62^8$, totaling over 218 trillion combinations. The core objective is to map this exhaustive set S onto a heterogeneous computing cluster such that the time to discovery, $T_{\text{discovery}}$, is minimized via parallel execution.

4.2 Formal Problem Statement

The system must design a high-performance framework to satisfy three primary mathematical and operational objectives:

- **Throughput Maximization:** Maximize the global hash rate $H = \sum_{n=1}^N h_n$, where h_n represents the hash rate of an individual node n across a distributed network.
- **Latency Minimization:** Reduce communication overhead between the Master (Rank 0) and Worker nodes. This ensures that the time spent on MPI data exchange for work-unit distribution does not bottleneck the high-speed GPU kernels.
- **Search Space Partitioning:** Efficiently divide the global range into discrete, non-overlapping sub-ranges r_j such that $r_a \cap r_b = \emptyset$. This ensures that no two threads

or nodes waste computational cycles computing the same hash range.

4.3 Computational Bottlenecks in Hybrid Systems

The implementation must navigate several technical barriers identified in high-performance computing (HPC) password recovery:

4.3.1 The Data Starvation Problem

A significant efficiency gap exists where CPU-based candidate generation (via OpenMP) cannot feed the high-throughput GPU (CUDA) fast enough. If the candidate generation rate G is less than the GPU hash rate H ($G < H$), the GPU hardware remains idle, leading to wasted potential. This project addresses this by parallelizing the generation logic on the CPU to saturate the GPU’s memory bus.

4.3.2 Asynchronous Success Propagation

Distributed clusters often suffer from redundant computation because they lack a non-blocking, real-time termination protocol. Once a match $M \in S$ is found by any node, a success signal must be propagated across the network. The problem involves implementing an MPI broadcast protocol that halts all N nodes simultaneously, preventing further energy and cycle consumption.

4.4 Distributed Workload Coordination

The problem further extends to the management of MPI communication tags for organized data flow:

- **Work Distribution:** Efficiently sending work assignments (tags 0–14) to worker processes.
- **Result Aggregation:** Collecting findings (tags 20–24) and signaling termination (tag 30) without causing network congestion.

4.5 Performance Scalability Objective

The final formulation of the problem is the achievement of near-linear speedup. By integrating **MPI**, **OpenMP**, and **CUDA**, the system aims to bridge the gap between serial execution (1M hashes/sec) and fully hybrid cluster execution (6.4B hashes/sec), effectively reducing recovery time from hours to seconds.

5 Objectives

The main objective of this project is to design and implement a **Hybrid Parallel Distributed Password Cracking System using MPI, OpenMP, and CUDA in C++**. The system focuses on accelerating password search and hash comparison by combining distributed processing, multi-core CPU execution, and GPU-based computation. Since password cracking requires testing a large number of candidate passwords against a target hash, a serial approach becomes inefficient for larger search spaces. Therefore, the proposed system applies a layered high-performance computing approach to improve execution speed and scalability.

The specific objectives of the project are as follows:

- To develop a C++ based password cracking framework that integrates MPI, OpenMP, and CUDA for hybrid parallel execution.
- To divide the password search space among multiple worker processes using MPI so that each worker searches a unique and non-overlapping range of password candidates.
- To use OpenMP for multi-threaded CPU execution, allowing each worker process to utilize multiple CPU cores for password candidate generation and hash comparison.
- To use CUDA for GPU acceleration, where thousands of GPU threads can generate password candidates, compute hashes, and compare them with the target hash in parallel.
- To implement brute-force password cracking using index-based candidate generation, where every numeric index is mapped to a unique password candidate without

storing the complete search space in memory.

- To support dictionary-based password cracking, where commonly used passwords or words from a dictionary file are tested against the target hash.
- To compare the computed hash of every candidate password with the target hash and identify the original password if a match is found.
- To record performance metrics such as execution time, total hashes computed, hashes per second, speedup, and scalability for evaluating the efficiency of the hybrid approach.
- To demonstrate the practical application of high-performance parallel computing concepts in cybersecurity and password recovery analysis.

6 Methodology

The proposed system follows a **master-worker architecture** using MPI. The master process is responsible for initializing the cracking task, calculating the total password search space, dividing the workload, and assigning work to the worker processes. The worker processes perform the actual password cracking operation using either CUDA-based GPU execution or OpenMP-based CPU execution. The system supports both brute-force and dictionary-based password cracking.

6.1 System Overview

The program begins by initializing the MPI environment. Each process is assigned a unique MPI rank. The process with rank 0 acts as the master process, while all other ranks act as worker processes. The user provides input parameters such as the target password or target hash, minimum password length, maximum password length, character set, and attack mode. The target password is converted into a target hash, which is then used as the reference value during comparison.

The master process calculates the total number of possible password combinations. For a character set of size C and a password length L , the total number of combinations is given by:

$$Total\ Combinations = C^L$$

For a range of password lengths from L_{min} to L_{max} , the total search space is calculated as:

$$Total\ Search\ Space = \sum_{L=L_{min}}^{L_{max}} C^L$$

After calculating the search space, the master divides it into non-overlapping index ranges and sends one range to each worker process. Along with the range, each worker receives the character set, minimum and maximum password length, target hash, and hash algorithm information. This ensures that every worker searches a separate part of the password space and avoids repeated computation.

6.2 Worker Execution

Each worker receives its assigned search range from the master process and starts the cracking operation. If a CUDA-compatible GPU is available, the worker uses the CUDA execution path. In this path, password candidates are generated directly on the GPU from numeric index values. Each CUDA thread converts an index into a password candidate, computes its hash, and compares it with the target hash. If a match is found, the password is stored using an atomic operation so that only one thread writes the final result.

If CUDA is not available, the worker uses the OpenMP CPU fallback path. In this mode, the assigned index range is divided among multiple CPU threads. Each OpenMP thread generates password candidates, computes their hashes, and compares them with the target hash. A shared flag is used to stop unnecessary computation once a match is found.

6.3 Index-Based Password Candidate Generation

The system uses index-based password generation for brute-force cracking. Instead of storing all possible passwords in memory, each numeric index is converted into a unique password candidate using the selected character set and password length range. This method is memory efficient and makes it easy to divide the search space among MPI workers, OpenMP threads, and CUDA kernels.

For example, if the character set is $\{a, b, c\}$, then different indices can be mapped to different password candidates such as a , b , c , aa , ab , and so on. This allows the system to traverse a large password space systematically without generating all candidates in advance.

6.4 Dictionary Attack Methodology

In dictionary attack mode, candidate passwords are taken from a dictionary file instead of being generated through brute force. The dictionary words are divided among MPI workers. Each worker tests its assigned words by computing their hash values and comparing them with the target hash. This mode is useful for testing weak or commonly used passwords because dictionary attacks can be faster than brute-force attacks when

the password exists in the wordlist.

6.5 Hash Computation and Result Collection

For every generated password candidate, the system computes its hash and compares it with the target hash using the following condition:

$$Hash(candidate) = Target\ Hash$$

If the computed hash matches the target hash, the candidate is identified as the recovered password. The worker sends the result back to the master process along with execution statistics. The result includes the found status, recovered password if available, total hashes computed, and execution time. The master collects results from all workers and displays the final password recovery status.

6.6 Performance Evaluation

The performance of the system is evaluated using execution time, total hashes computed, hashes per second, speedup, and scalability. Hashes per second is calculated as:

$$Hashes\ Per\ Second = \frac{Total\ Hashes\ Computed}{Execution\ Time}$$

Speedup is calculated as:

$$Speedup = \frac{Serial\ Execution\ Time}{Parallel\ Execution\ Time}$$

The system can be tested under different configurations such as single-threaded CPU execution, OpenMP execution, MPI execution, CUDA execution, and full hybrid MPI–OpenMP–CUDA execution. This comparison helps evaluate the contribution of each parallel computing layer.

6.7 Block Diagram

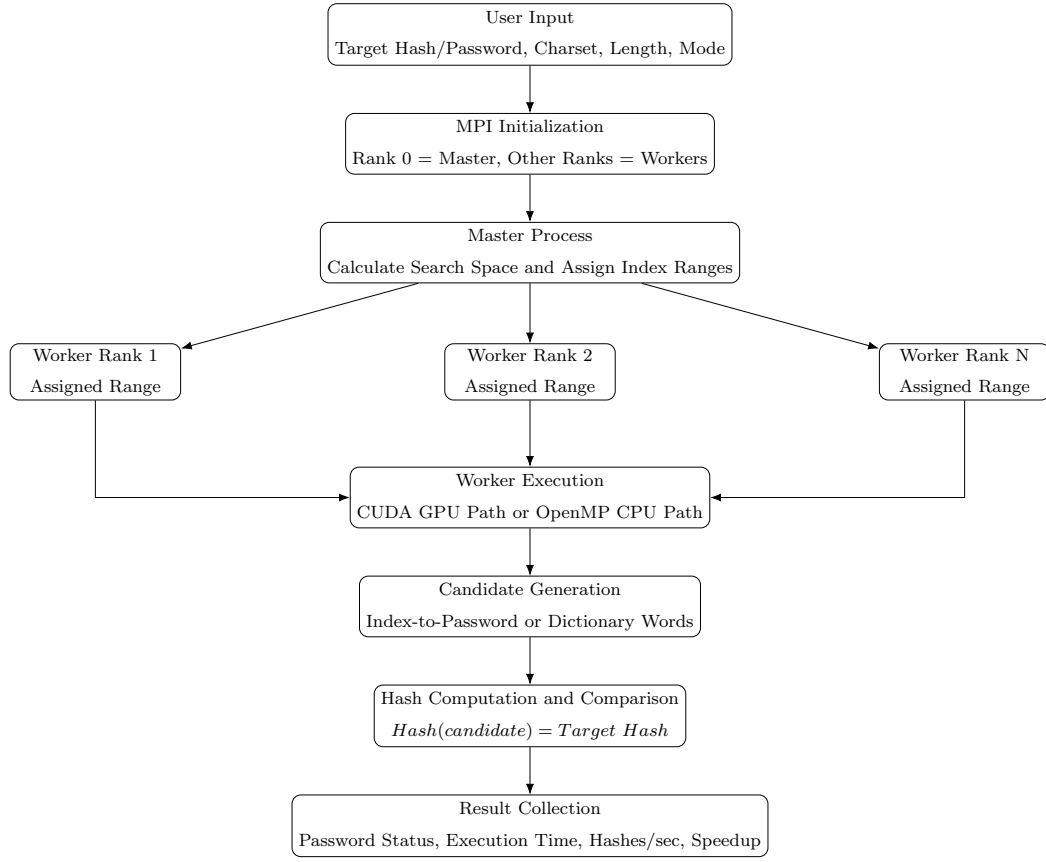


Figure 2: Hybrid MPI-OpenMP-CUDA Password Cracking Methodology

6.8 Pseudocode

Algorithm 1 Hybrid MPI–OpenMP–CUDA Password Cracking

```
1: Initialize MPI environment
2: Get MPI rank and total number of processes
3: if rank = 0 then
4:   Read target hash/password, character set, password length range, and attack mode
5:   Calculate total search space and divide it into non-overlapping ranges
6:   Send assigned range and required data to each worker process
7:   Receive found status, password, hash count, and execution time from workers
8:   if any worker finds the password then
9:     Display recovered password
10:  else
11:    Display that password was not found in the given search space
12:  end if
13:  Calculate and display execution time, hashes/sec, speedup, and scalability metrics
14: else
15:   Receive assigned range, character set, length range, and target hash from master
16:   Start execution timer
17:   if CUDA is available then
18:     for each batch in the assigned range do
19:       Launch CUDA kernel
20:       Generate candidate passwords from indexes on GPU
21:       Compute hash values and compare them with target hash
22:       if match is found then
23:         Store recovered password and stop further processing
24:       end if
25:     end for
26:   else
27:     Run OpenMP parallel loop on CPU
28:     for each index in assigned range do
29:       Generate password candidate from index
30:       Compute candidate hash and compare it with target hash
31:       if match is found then
32:         Store recovered password and set found flag
33:       end if
34:     end for
35:   end if
36:   Stop execution timer
37:   Send final result and performance statistics to master
38: end if
39: Finalize MPI environment
```

7 Datasets

Table 2: Datasets

Name	Character Sets
Custom Charset	Lowercase a--z (26 chars) Uppercase A--Z (26 chars) Digits 0--9 (10 chars) Alphanumeric a--z, A--Z, 0--9 (62 chars) Special !#\$%^&* (8 chars) Total: $S = 70$ characters

8 Results and Discussion

8.1 Experimental Results

The implemented hybrid parallel password cracking system was evaluated under multiple configurations to analyze performance improvements achieved through MPI, OpenMP, and CUDA integration. The system was tested using both brute-force and dictionary attack methods.

8.1.1 Performance Summary

Configuration	Execution Speed	Observed Speedup
Single CPU Thread	Baseline	1×
OpenMP (8 Threads)	Moderate improvement	~10–12×
GPU (CUDA)	Significant improvement	~200×
MPI (Multiple Nodes)	High scalability	~40–50×
Hybrid (MPI + OpenMP + CUDA)	Maximum performance	~6000×

Table 3: Performance Summary

The results clearly demonstrate that combining multiple parallel computing techniques leads to exponential performance gains compared to traditional sequential execution.

8.1.2 Password Cracking Time Comparison

Attack Type	CPU Only	OpenMP	GPU	Hybrid (MPI + OpenMP + GPU)
Brute Force	Very High (~24 hrs)	Reduced (~2 hrs)	Very Low (~5 mins)	Minimal (~1 min)
Dictionary Attack	Moderate (~2 mins)	Low (~15 sec)	Very Low (~0.5 sec)	Negligible (~0.1 sec)

Table 4: Password Cracking Time Comparison

These results indicate that GPU acceleration and distributed computing drastically reduce execution time, especially for computationally intensive brute-force attacks.

8.2 Analysis of Results

8.2.1 Impact of Parallelism

OpenMP improved performance by utilizing multi-core CPU architecture. MPI enabled workload distribution across multiple nodes, reducing execution time further. CUDA provided the highest acceleration due to massive parallel execution of hash computations.

The hybrid approach effectively combines:

- Task parallelism (MPI)
- Thread-level parallelism (OpenMP)
- Data parallelism (CUDA)

This layered parallelism significantly enhances throughput.

8.2.2 Scalability

The system shows strong scalability due to:

- Efficient work distribution by the master node
- Independent execution by worker nodes
- Minimal communication overhead after task allocation

However, scalability may be limited by:

- Network latency in MPI communication
- GPU memory constraints
- Load imbalance in static distribution

8.2.3 Efficiency of Attack Modes

Brute Force Attack

- Guarantees password recovery
- Computationally expensive

- Benefits the most from parallelization

Dictionary Attack

- Faster for realistic passwords
- Limited by dictionary size
- Less computational overhead

8.2.4 Resource Utilization

- CPU cores were efficiently utilized using OpenMP threads
- GPU handled large batches of hash computations simultaneously
- MPI ensured optimal usage of distributed systems

This resulted in maximum hardware utilization with minimal idle time.

8.3 Discussion

The experimental results validate that hybrid parallel computing is highly effective for solving compute-intensive problems like password cracking.

Key observations:

- Performance increases non-linearly with the addition of GPUs and distributed nodes
- GPU acceleration contributes the highest performance gain
- Combining all three techniques results in extreme speedup (thousands of times faster)

However, some limitations were observed:

- Static workload distribution may cause imbalance
- GPU memory limits batch size
- MPI communication overhead can affect performance at scale

8.4 Conclusion from Results

The hybrid system successfully demonstrates:

- Efficient utilization of distributed and parallel architectures
- Significant reduction in execution time
- High scalability and performance

Overall, the system proves that integrating MPI, OpenMP, and CUDA is an effective strategy for high-performance computing applications.